

Principles of Compiler Construction

Prof. Wen-jun LI

School of Computer Science and Engineering

lnslwj@mail.sysu.edu.cn

© Copyright 2008-2022, Sun Yat-sen University

Lecture 2. Formal Languages

1. **What** Is a Language, Formally?
2. **How** to Define a Specific Language?
3. Context-Free Grammar
4. Parse Tree
5. Ambiguity
6. Chomsky Hierarchy

Formal System

- Don't care the meaning of symbols.
  - Symbols can be substituted.
- Only depend on formally defined rules.
- Programs are formal systems.

1. What Is a Language, Formally?

- Alphabet ::= a set of symbols 符号的集合 数学和代码都需要考虑到边界问题
  - Non-empty and finite 非空有限
  - Symbol is a meta-definition 从alphabet中拿出的序列
- String (sentence or word) ::= a sequence of symbols which are from the alphabet 必须是有穷的，但可以是空的
  - Finite; Permit empty string ε [*epsilon*]
  - Manipulation: empty, length, prefix, suffix, substring, proper 真子集, subsequence, concatenation, Kleene star (closure)
- Σ^* :: = the set of all possible strings on Σ
  - Mathematically, a monoid (semigroup with an identity) generated by Σ.

封闭性、二元运算带结合性、单位元

Formal Definition of Language

- Language ::= A set of strings
  - Maybe infinite, or an empty set {} (or ∅)
- Language manipulation
  - Union: $L_1 \cup L_2$
  - Concatenation: $L_1 L_2$
  - Exponential: $L^0 = \{\varepsilon\}$, $L^1 = L$, $L^2 = LL$, ...
  - Kleene closure (star): $L^* = \bigcup_{i=0}^{\infty} L^i$
  - Positive closure (plus): $L^+ = \bigcup_{i=1}^{\infty} L^i = L^* L$
- What is a meta-definition?

Mapping (Binding) to Practice

- Binding
  - Abstract: formal definition of a language
  - Concrete: programming languages
- **Discussion**
  - For C programs
    - Alphabet Σ → ? ASCII码
    - Σ^* → ? 拿ASCII码写出的所有的串
    - Language → ? 拿ASCII码写出的合乎语法的串
    - A string in the language → ? 一个良构的C语言程序
  - For Java programs
    - Alphabet Σ → ? Unicode
    - A string in the language → ? 一个良构的Java语言程序

2. How to Define a Specific Language?

- Essentially, how to define a set:
  - Enumeration
    - { aa, ab, ba, bb }
  - Partial enumeration
    - { a, ab, abb, abbb, ... }
  - Predicate description
    - $\{ x \mid \mathbf{P}(x) \wedge x \in \Sigma^* \}$
- Problem: how to define the predicate **P**?
  - Described (by an expression)
  - Generated (by a grammar)
  - Recognized (by an automaton)

3. Context-Free Grammar (CFG)

上下文无关语法！

- CFG is a 4-tuple (Σ, N, P, S)
  - Σ: alphabet, set of terminals
  - $N \cap \Sigma = \varnothing$: set of nonterminals
  - P: set of rewriting rules (productions)
    - Each production has the form $A \rightarrow \alpha$, where $A \in N \wedge \alpha \in (\Sigma \cup N)^*$ 重写规则
  - $S \in N$: start (goal) symbol

非终止符号不能是终止符号里的符号！
起始符号必须为非终止符号

An Example

- A Context-Free Grammar
  - G_1 = ({a, b, c}, {A, B}, {A→aB, A→bB, A→cB, B→a, B→b, B→c}, A)
- Brief notation

  A → aB | bB | cB

  B → a | b | c

左边的为开始符号

这样子是有二义性的！
（不知道哪里是开始符号，不知道哪里是终结符号）

约定：
所有的大写字母为非终结符号
所有的小写字母为终结符号

Derivation

- Derive (rewrite) 重写 双线箭头为推导！分清楚单双线！
  - $A \Rightarrow bB \Rightarrow bc$ 根据产生式置换成产生式的右部
  - Direct derivation: $bB \Rightarrow bc$
  - n-step derivation: $A \Rightarrow^{2} bc$
  - 0 or more steps derivation: $A \Rightarrow^{*} bc$ 星号可能右边和左边相等！推导次数的写法
  - 1 or more steps derivation: $A \Rightarrow^{+} bc$
  - Left-most derivation: $A \Rightarrow^{*}_{lm} bc$
    - Every step must be left-most derivation
  - Right-most derivation: $A \Rightarrow^{*}_{rm} bc$
    - Every step must be right-most derivation
  - Canonical derivation: right-most

经典的推导：最右推导

Binding to Practice

- Arithmetic expressions

  $E \rightarrow E + T \mid T$

  $T \rightarrow T * F \mid F$

  $F \rightarrow (E) \mid n$

Each nonterminal represents a subset of the language

- Derivation of **n+n*n**
  - $\mathbf{E} \Rightarrow E+\mathbf{T} \Rightarrow \mathbf{E}+T*F \Rightarrow \mathbf{T}+T*F \Rightarrow F+\mathbf{T}*F \Rightarrow F+F*\mathbf{F} \Rightarrow F+\mathbf{F}*n \Rightarrow \mathbf{F}+n*n \Rightarrow n+n*n$
  - Not canonical!
- Canonical derivation of **n+n*n**

每次都用的是式子中最右边的非终结字符去做推导

  - $\mathbf{E} \Rightarrow E+\mathbf{T} \Rightarrow E+T*\mathbf{F} \Rightarrow E+\mathbf{T}*n \Rightarrow E+\mathbf{F}*n \Rightarrow \mathbf{E}+n*n \Rightarrow \mathbf{T}+n*n \Rightarrow \mathbf{F}+n*n \Rightarrow n+n*n$

能预测的就很简单，但是不能预测的就需要回溯，就很麻烦！

Reduction

- Reduce
  - Derive in reverse order
  - Direct reduction
  - n-step reduction
  - Left-most (canonical) reduction
  - Right-most reduction

最右推导和最左规约是一对的（逆运算）

临时难点！

Sentences and Handles

- Sentential form
  - If $S \Rightarrow^* \alpha$, α is a sentential form of G.
  - Right (Left) sentential form
- Sentence
  - A sentential form with no nonterminals.
- Phrase
  - If $S \Rightarrow^* \alpha A \omega \wedge A \Rightarrow^+ \beta$, β is a phrase of $\alpha\beta\omega$ on A.
- Simple phrase
  - If $S \Rightarrow^* \alpha A \omega \wedge A \Rightarrow \beta$, β is a simple phrase.
- Handle
  - If $S \Rightarrow^*_{rm} \alpha A \omega \Rightarrow_{rm} \alpha\beta\omega$, $A \rightarrow \beta$ in the position following α is a handle of $\alpha\beta\omega$.

Define a Language by Grammar

注意P4和P8的定义

- Language defined by a given CFG G
  - $L(G) = \{\alpha \mid S \Rightarrow^* \alpha \wedge \alpha \in \Sigma^*\}$ 上边这里如果*换+号就不会有影响了
  - That is the set of all sentences of G
- Set of all sentential forms of G is
  - $SF(G) = \{\alpha \mid S \Rightarrow^* \alpha \wedge \alpha \in (\Sigma \cup N)^*\}$
- Thus we have $L(G) = SF(G) \cap \Sigma^*$.

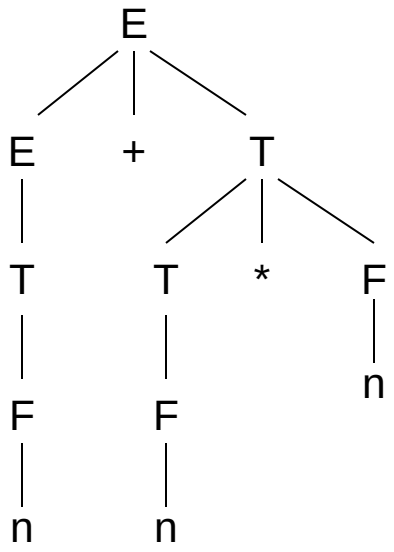
BNF (Backus-Naur Form)

语言需要结合上下文去理解

- Equivalent to CFG
  - Change "$\rightarrow$" to "::="
  - De facto standard to define a computer language
- EBNF (Extended BNF)
  - ISO/IEC 14977:1996(E). *The Standard Metalanguage Extended BNF*.
  - =, "terminals", [optional], {repetition}, (group), (*comment*), etc.

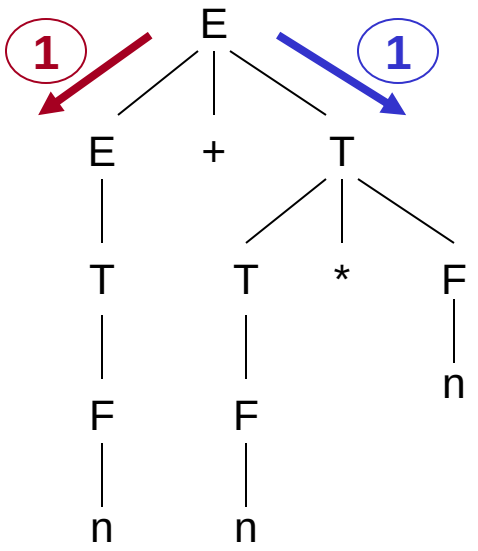
4. Parse Tree

- Graphic representation of derivations
  - Root: start symbol
  - Leafs: terminals or ε
  - Interior nodes: nonterminals
  - Offspring of a nonterminal: a production
- An example
  - **E** ⇒ E+**T** ⇒ E+T*__F__
    ⇒ E+**T***n ⇒ E+**F***n
    ⇒ **E**+n*n ⇒ **T**+n*n
    ⇒ **F**+n*n ⇒ n+n*n



Insight into a Parse Tree

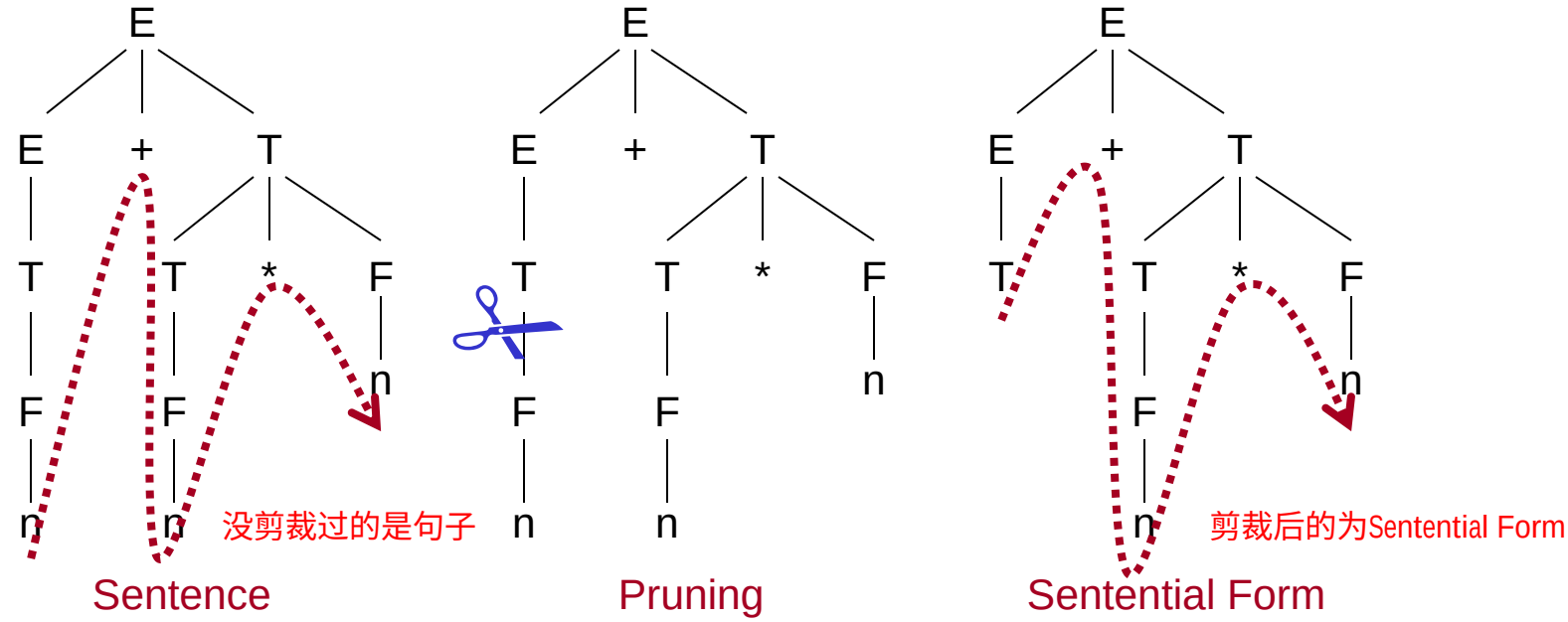
- An **abstraction** of derivations
  - Derivation order is discarded.



推导的次序被忽略（抽象化）

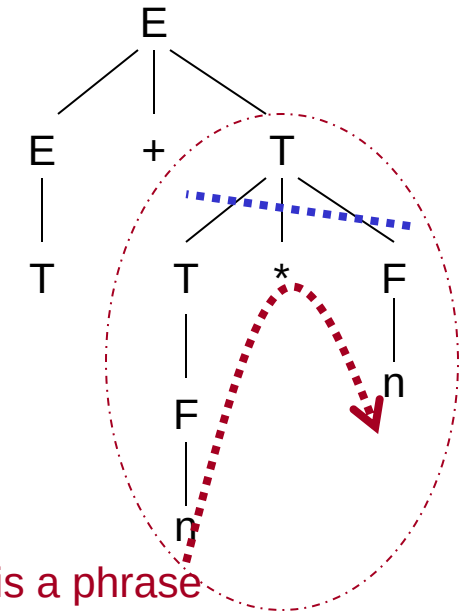
Sentences and Sentential Forms in a Parse Tree

- Sentence
  - Frontier of the parse tree, e.g. n+n*n
- Sentential Form
  - Frontier of the pruned parse tree, e.g. T+n*n



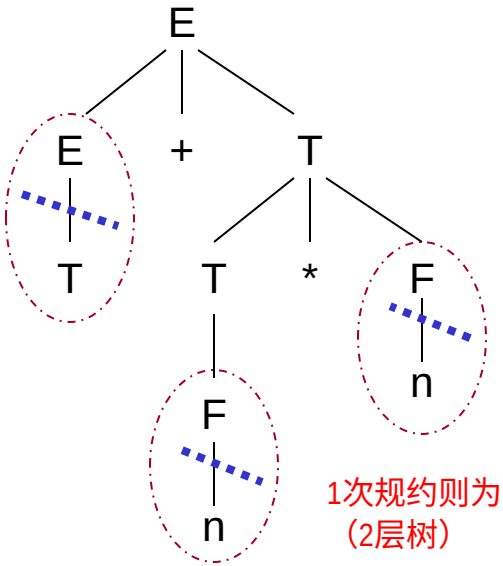
Phrases and Handles in a Parse Tree

- For a given sentential form, e.g. T+n*n
  - Phrase: frontier of a nonterminal subtree
  - Simple phrase: frontier of a 2-level subtree
  - Handle: left-most simple phrase



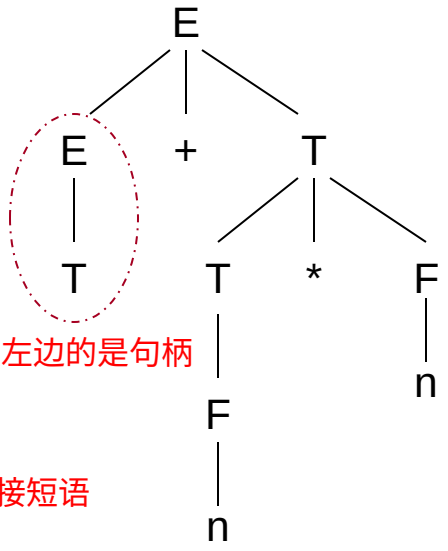
n*n is a phrase

关于T的短语



The only simple phrases are **T**, **n**, and **n**

1次规约则为直接短语（2层树）



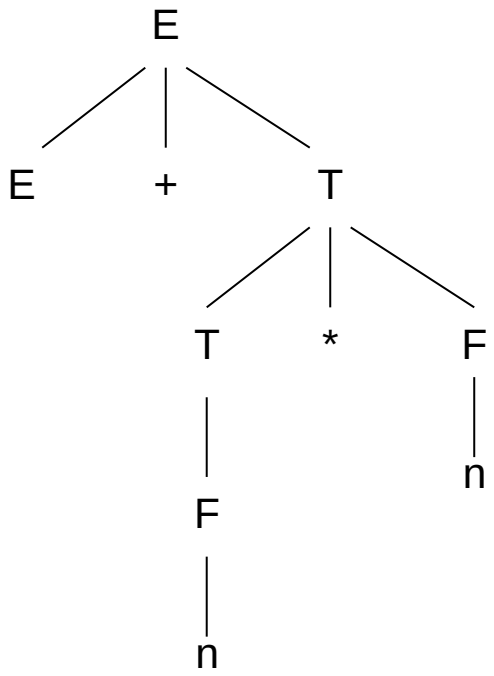
最左边的是句柄

Handle is unique: **T**

裁剪的时候必须要是完整的子树！！！

Discussions

- What is ...
  - The sentential form
  - All simple phrases
  - The only one handle



Parsing

- There are 2 questions that must be answered by parsing
  - Is the source program well-formed (legal in syntax)?
    - That is, is it a sentence of the language?
  - If it is, what's the syntax construction of the program?
    - Tree is an ideal data structure to present **part-of** relationship.
- 2 parsing strategies
  - Top-down vs. bottom-up

Top-Down Parsing

- Beginning at the start symbol
  - Usually expanding nonterminals in depth-first manner (predictive in nature)
  - Left-most derivation
  - Pre-order traversal of the parse tree
- Example: LL(k)
  - Read from **L**eft, and **L**eft-most derivation, with **k** lookaheads
  - Recursive descent (predictive) parsing

从根节点开始构造子树得到叶节点

这里对应的是最左推导

Bottom-Up Parsing

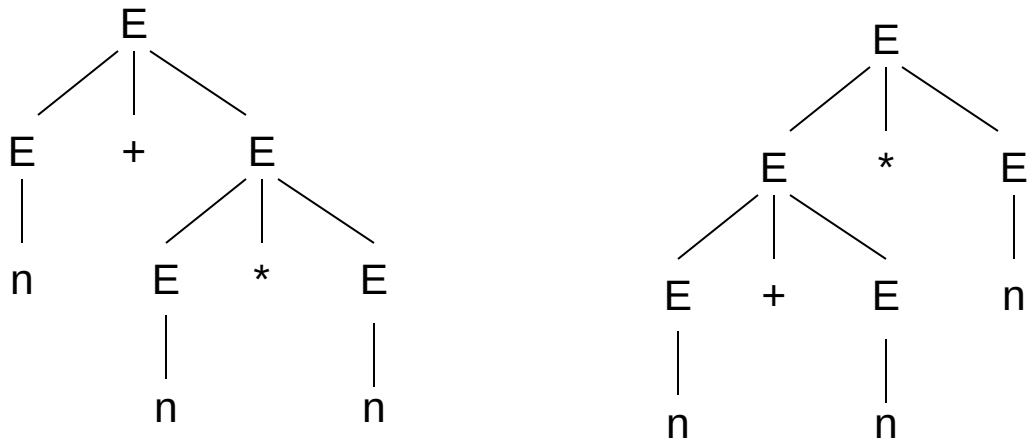
- Beginning from the terminal input string
  - Determining the production used to generate leaves
  - Right-most derivation in reverse order
  - Post-order traversal of the parse tree
- Example: LR(k)
  - Read from **L**eft, and **R**ight-most derivation, with **k** lookaheads
  - Parser generator yacc

先摊开叶节点，再构造子树

（做的事情是规约，即最左规约，因此这里对应的是最右推导）

5. Ambiguity

- A grammar with more than one parse tree for at least one sentence in the language
  - E → E + E | E * E | (E) | n



2 parse trees for the sentence **n + n * n**

两个句子是一样的，但是两个树是不一样的（二义性）

这里的原因是因为文法没有规定！

Ambiguous: Grammar vs. Language

- Ambiguous Language (inherent)
  - There exists no unambiguous grammar to define the language.
- Ambiguous Grammar (postnatal)
  - May be transformed to an equivalent unambiguous grammar.
  - The following two grammars define the same language
    - E → E + E | E * E | (E) | n
    - E → E + T | T
      T → T * F | F
      F → (E) | n

同一种文法既可以无二义，也可以二义，则称为后天二义

Binding to Practice

程序设计语言里主要是if语句的二义性

- Ambiguity in Programming Languages
  - Operators in an expression
  - Dangling **`else`** in embedded **`if`** statements
- Both of them are postnatal
  - Can be removed by additional disambiguation rules
    - Precedence and associativity
    - Inner "**`if`**" matched
  - Two approaches to resolving ambiguities
    - Rewrite the ambiguous grammar
    - Ad hoc solution: application of additional rules

6. Chomsky Hierarchy

Useful in Practice

Useful in Theory

Class	Grammar	Restriction	Recognizer
3	Regular 正则语法 产生式只有两种形态	$A \to aB$ or $A \to a$, where $A, B \in N \wedge a \in \Sigma \cup \{\varepsilon\}$. $A \to \varepsilon$ permitted if A is the start symbol and does not appear on the right of any production.	Finite-State Automaton (**FSA**)
2	Context-Free	$A \to \alpha$, where $A \in N \wedge \alpha \in (\Sigma \cup N)^*$.	Push-Down Automaton (**PDA**)
1	Context-Sensitive	$\alpha \to \beta$, where $\alpha, \beta \in (\Sigma \cup N)^* \wedge \alpha \neq \varepsilon \wedge \lvert\alpha\rvert \leq \lvert\beta\rvert$. β can't be ε, unless α is the start symbol and does not appear on the right of any production.	Linear-Bounded Automaton (**LBA**)
0	Unrestricted	$\alpha \to \beta$, where $\alpha, \beta \in (\Sigma \cup N)^* \wedge \alpha \neq \varepsilon$.	Turing Machine (**TM**)

Regular Language

- Ability of regular languages
  - What is it able to define?
    - Identifiers
    - Decimal constants
  - What isn't it able to define?
    - Matched parentheses
- Satisfy the requirement of lexical descriptions

Context-Free Language

- Ability of context-free languages
  - What is it able to define?
    - Matched constructs
  - What isn't it able to define?
    - Only use declared variables
    - Matched parameter passing
- Satisfy the requirement of syntactic descriptions

Exercise 2.1

- Given the following grammar:

$$S \rightarrow \mathbf{(}\ L\ \mathbf{)} \mid \mathbf{a}$$

$$L \rightarrow L\ \mathbf{,}\ S \mid S$$

Construct a parse tree for the sentence
(a, ((a, a), (a, a)))

Exercise 2.2

- Given the following grammar:

  bexpr → bexpr **or** bterm | bterm

  bterm → bterm **and** bfactor | bfactor

  bfactor → **not** bfactor | **(** bexpr **)** | **true** | **false**

  Construct a parse tree for the sentence
  not (true or false)

Exercise 2.3

- Is the grammar:
  $S \rightarrow a\ S\ b\ S \mid b\ S\ a\ S \mid \varepsilon$
  ambiguous? Why?

Further Reading

- Dragon Book, 2nd Edition (DBv2)
  - Comprehensive reading:
    - Section 2.1 – 2.3, 4.2.1 – 4.2.6 for the concepts of CFG, derivation, parse tree and ambiguity.
    - Section 4.5.1 – 4.5.2 for the concepts of reduction and handle.
  - Skip reading:
    - Section 2.4 for top-down parsing.
- Skip reading: definition of languages
  - BNF or EBNF of Oberon-0, Oberon, OMG IDL, and Lua from our course website
- Skip reading: EBNF standard
  - ISO/IEC. The Standard Metalanguage EBNF.

Enjoy the Course!

